AdaBoost: How It Works

Contents

1. Overview . . . 2
    1.1 Why learn AdaBoost . . . 3
2. Three ideas we need first . . . 3
    2.1 Weak learners . . . 3
    2.2 Decision stumps . . . 4
    2.3 Labels +1 and -1 . . . 5
3. A stage-wise additive model . . . 5
4. Stage by stage . . . 6
    4.1 Stage 1: the first stump . . . 6
    4.2 Passing the mistakes on . . . 7
    4.3 The say of each stump: alpha . . . 7
    4.4 Stages 2 and 3 . . . 7
5. The final prediction: a weighted vote . . . 8
6. Why it works: the combined boundary . . . 9
7. Summary . . . 11
8. Sources . . . 11
9. Key terms . . . 12

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Underfitting; Hyperparameter tuning; Bias-variance trade-off; Decision trees; Ensemble learning.
- **Leads to:** Gradient boosting.
- **Compare with:** Bagging; Gradient boosting; Stacking and blending.

1. Overview

Key point: AdaBoost trains weak models one after another. Each one pays more attention to the observations the previous one got wrong, and each gets a say based on how accurate it was. The final answer is a weighted vote.

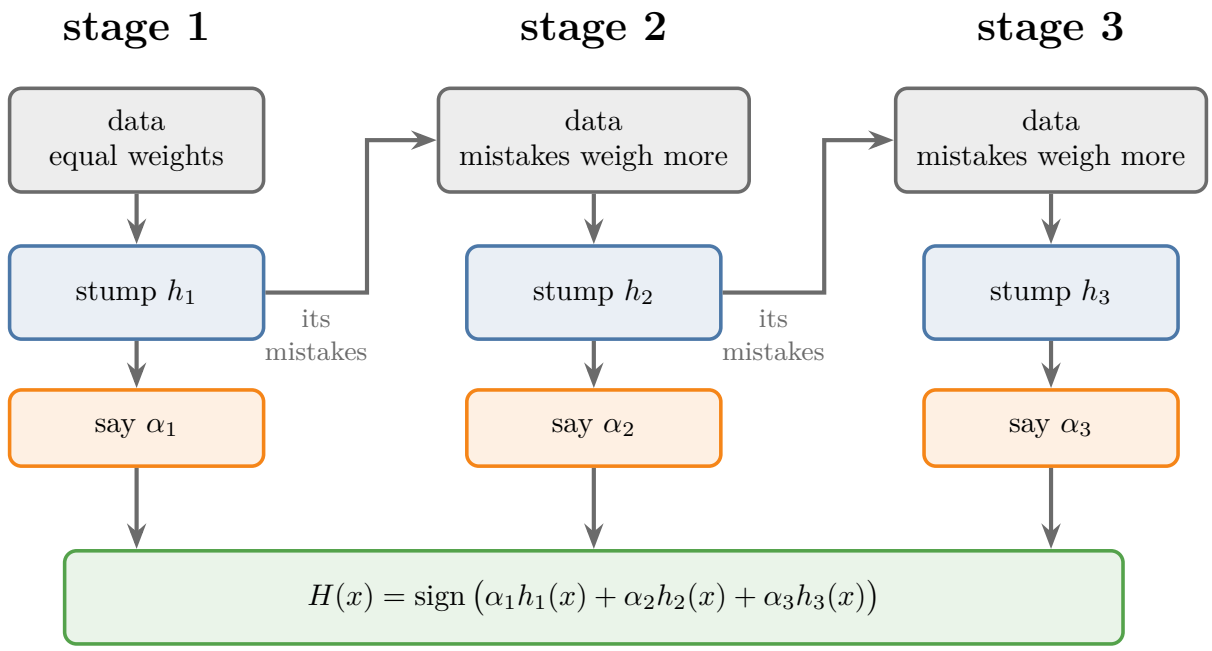


Figure 1: AdaBoost as a stage-wise additive model: each stage trains a stump on reweighted data and gets a say; the final model is the sign of the weighted sum

AdaBoost (G-167) is the first **boosting** (G-318) algorithm we study. Boosting was previewed in the four types of ensemble: models trained in series, each one focusing on the mistakes of the one before. Figure 1 shows how AdaBoost does it.

Compared with a **random forest** (G-1611; many full trees trained independently and voting, see how a random forest works), AdaBoost changes three things:

	Random forest	AdaBoost
Base model	full-size decision trees	**decision stumps**: trees with one split (section 2.2)
Vote	every tree's vote counts the same	each stump has its own **say**, larger when it made fewer mistakes (section 4.3)
Order	trees are built independently; order does not matter	stumps are built in sequence; each one is shaped by the mistakes of the one before (section 4.2)

This Note gives the core idea for classification, on a picture. A worked example fills in the numbers: how the say of each model is computed, and how the mistakes are passed on.

> **Extra:** The name is short for **Adaptive Boosting**: each stage adapts to the mistakes of the stages before. Yoav Freund and Robert Schapire published it in 1995 (Freund and Schapire, 1997). One of its first big uses was fast face detection (Viola and Jones, 2001). Deep learning has since taken over that job.

1.1 Why learn AdaBoost

> **Key point:** Other boosting algorithms often score higher, but **AdaBoost** (G-167) is the simplest, and it teaches the ideas they all build on.

AdaBoost gives good results, but in practice random forests, **gradient boosting** (G-859) and XGBoost often beat it. We study it anyway because it is the entry point to **boosting** (G-318). Once AdaBoost is clear, about half of boosting is clear, and gradient boosting and XGBoost become much easier.

2. Three ideas we need first

> **Key point:** A weak learner is barely better than guessing; a decision stump is a tree with one split; **AdaBoost** (G-167) labels the two classes +1 and -1.

2.1 Weak learners

> **Key point:** A weak learner is a model whose accuracy is only a little above 50%.

On a two-class problem, guessing at random is right about 50% of the time, like calling heads or tails on a coin toss. A **weak learner** (G-2104) is a model whose accuracy is just above that: 55%, say. A **strong learner** (G-1904) is a model with high accuracy, 95% or 99%.

AdaBoost (G-167) builds one strong model by combining many weak learners. Each one alone is poor; together they are good.

2.2 Decision stumps

> **Key point:** A decision stump is a decision tree with `max_depth=1`: one question, one split, two regions.

A **decision stump** (G-559) is a decision tree whose maximum depth is 1. It asks a single question, such as "is CGPA at most 6.75?", and splits the data into two regions. A tree of depth 1 was shown to underfit (it is too simple to capture the pattern). Here that weakness is exactly what we want: a stump is a **weak learner** (G-2104). A model that already gets every training observation right leaves no mistakes to pass on, so boosting would stop after it (the rule of thumb tests this with fully grown trees).

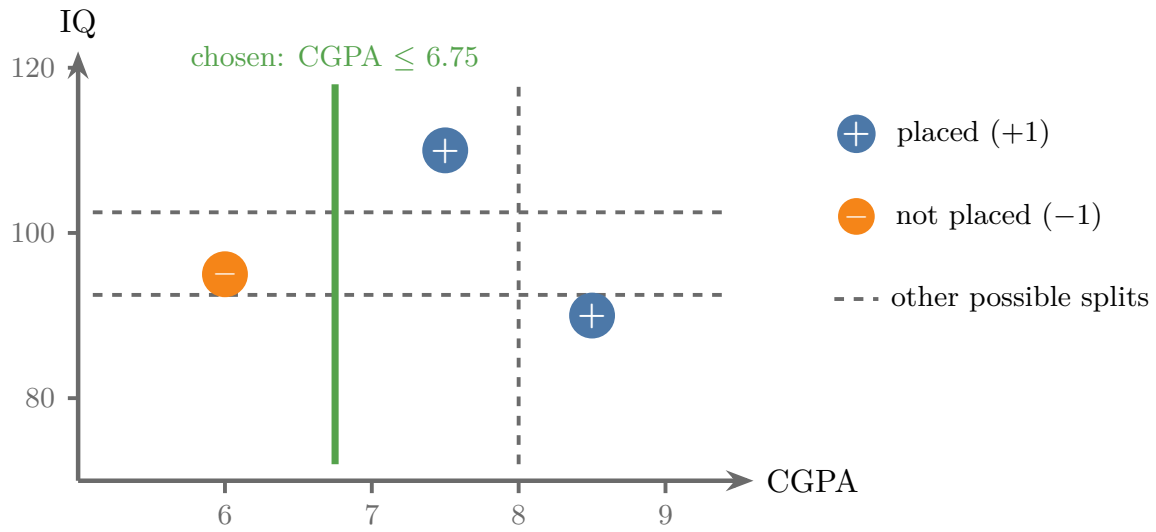


Figure 2: Three students on CGPA and IQ: every possible stump is one cut parallel to an axis; the chosen cut gives the largest information gain

Figure 2 shows three students: two placed (+), one not placed (-).

- A full decision tree can make many cuts, as in the geometry of a decision tree.
- A stump makes exactly one cut. It is either parallel to the IQ axis (a CGPA threshold) or parallel to the CGPA axis (an IQ threshold).
- With three points there are only a few candidate cuts: one between each pair of neighbouring values, on each axis.

The stump tries every candidate and keeps the one with the largest **information gain** (G-946; how much a split makes the groups purer, see the definition). In Figure 2, the cut at CGPA 6.75 separates the classes perfectly, so it wins.

AdaBoost (G-167) can use any algorithm as its weak learner, even a neural network. In practice it almost always uses decision stumps, for two reasons:

1. they give the best results with AdaBoost;

2. each one is easy to read, so we can see what every stage does.

2.3 Labels +1 and -1

> **Key point:** In **AdaBoost** (G-167) the positive class is +1 and the negative class is -1, not 1 and 0.

Most classifiers name the two classes 1 and 0. AdaBoost uses **+1 and -1**: placed students are +1, students not placed are -1. Section 5 shows why: the final vote adds up the models' answers, and a -1 pulls the total down, while a 0 would add nothing.

3. A stage-wise additive model

> **Key point:** "Additive": the final model is a sum of **weak learners** (G-2104). "Stage-wise": they are added one at a time, in order.

AdaBoost (G-167) is often described as a **stage-wise additive method** (a **stage-wise additive model**, G-1867):

- **additive:** the final model adds up several weak learners;
- **stage-wise:** the weak learners are added one after another, in sequence, each in its own stage.

In Figure 1, each column is a stage. The arrow between the columns is what makes it **boosting** (G-318): each stage hands its mistakes to the next.

Figure 3 shows the "additive" part on the 10 students of section 4: the vote of the first stump alone, of the first two, and of all three. Each added stump changes the vote only where its say tips the sum.

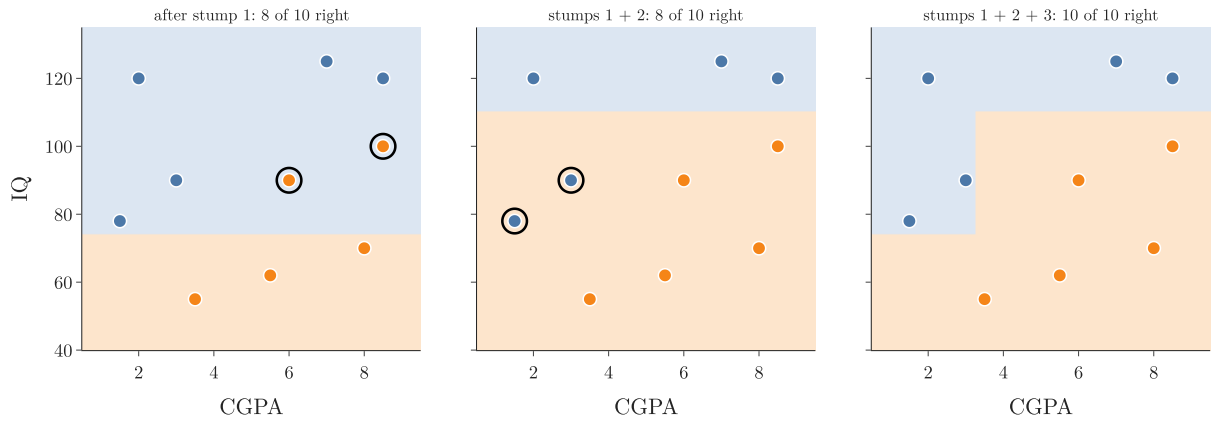


Figure 3: The vote of the first 1, 2 and 3 stumps on the 10 students of section 4 (blue region: placed; circled: wrong). One stump gets 8 right, two stumps still 8, and the sum of all three gets all 10 right

4. Stage by stage

> **Key point:** In every stage: fit a stump, find its mistakes, give the stump a say, then make its mistakes heavier for the next stage.

We follow 10 students. Each student is an **observation** (G-1374; one record, one row of the data table). Each has two **features** (G-772; input variables, one column each), CGPA and IQ, and a **target** (G-1949; the output we predict): placed (+1) or not placed (-1). Figure 4 animates the three stages.

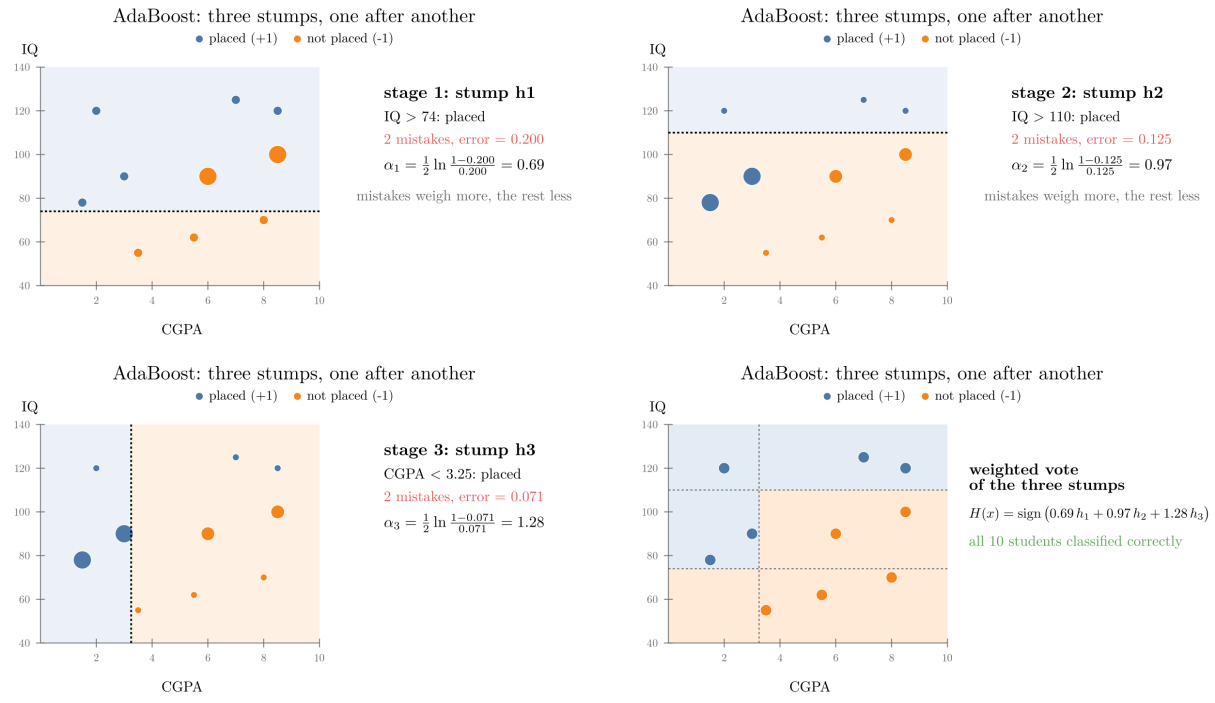


Figure 4: AdaBoost on 10 students. In each stage the dashed line is the stump's split; the dot size shows each student's weight, which grows after a mistake and shrinks after a correct answer. The last panel is the weighted vote of the three stumps

4.1 Stage 1: the first stump

> **Key point:** The first stump is the single cut with the largest information gain; it gets some students wrong.

Many stumps are possible on this data. As in section 2.2, we keep the one with the largest information gain: **IQ above 74 means placed** (Figure 4, stage 1). Above the line is the placed region, below it the not-placed region.

The stump does its job, but not perfectly: two students who were not placed sit in the placed region. These are its **mistakes**.

4.2 Passing the mistakes on

> **Key point:** Before the next stage, the misclassified observations are made more important, so the next stump concentrates on them.

The first stump tells the next one, in effect: "I got these students wrong; take extra care with them." **AdaBoost** (G-167) does this by raising the **importance** of the misclassified observations in the data and lowering that of the rest.

In Figure 4 this is the dot size: the two mistakes grow, the eight correct students shrink. The exact amounts, and the technique (called **upsampling**, G-2063), are the subject of step 7: upsampling.

4.3 The say of each stump: alpha

> **Key point:** Each stump gets a weight, alpha, from how few mistakes it made. Good stumps get a big say in the final vote, poor ones a small say.

At the end of each stage we also compute a number α (alpha) for the stump. Alpha, the **model weight** (G-192), sets how much **say** the stump will have in the final prediction. Few mistakes give a large α; many mistakes give a small one. The formula is in the say of the stump.

The say is where **boosting** (G-318) differs from bagging. In bagging (the core idea), every base model's vote counts the same, like a democracy. In boosting, each model's vote is weighted by how well it performed.

4.4 Stages 2 and 3

> **Key point:** Each new stump fixes the previous stump's mistakes but makes some of its own, which are passed on in turn.

Stage 2. The data now has heavier weights on the two missed students. A new stump is fitted: **IQ above 110 means placed** (Figure 4, stage 2). The new stump gets those two students right, but it makes two new mistakes: two placed students fall in its not-placed region. Again we compute its say, α_2, and make its mistakes heavier.

Stage 3. The next stump is **CGPA below 3.25 means placed** (Figure 4, stage 3). This third stump fixes stage 2's mistakes and makes two of its own. We compute α_3.

We stop at three stumps here. With more stages the process simply repeats.

Stage	Stump	Mistakes	Weighted error	Say α
1	IQ $>$ 74: placed	2	0.200	0.69
2	IQ $>$ 110: placed	2	0.125	0.97
3	CGPA $<$ 3.25: placed	2	0.071	1.28

The weighted error is the share of the total weight that sits on the misclassified observations. Here the weighted error falls from stage to stage because each new stump's two mistakes are students that every earlier stump got right, so the reweighting has made them light; the weighted error in step 3 computes it.

5. The final prediction: a weighted vote

> **Key point:** Multiply each stump's answer (+1 or -1) by its **alpha** (G-192), add, and take the sign: positive means +1, negative means -1.

After training we have three stumps and three alphas. We write each stump as a function $h_t(x)$, a **hypothesis function** (G-912): it takes a student's features x and returns +1 or -1.

1. **In words:** each stump votes +1 or -1; each vote is multiplied by that stump's say; we add the results; the sign of the total is the prediction.

2. **Formula:** with T stumps,

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \, h_t(x)\right)$$

   For three stumps this is:

$$H(x) = \text{sign}\Big(\alpha_1 h_1(x) + \alpha_2 h_2(x)$$

$$+\alpha_3 h_3(x)\Big)$$

   Here $\sum_{t=1}^{T}$ means "add the term for stump 1, 2, up to T". The **sign** (the **sign function**, G-1800) of a number is +1 if it is positive and -1 if it is negative.

3. **Example:** to see the vote on its own, take a made-up set of three stumps (not the three of section 4) and one new student. Suppose the stumps say $h_1 = -1$ (not placed), $h_2 = +1$ (placed), $h_3 = -1$, and their alphas are 2, 10 and 1:

$$2 \times (-1) + 10 \times (+1) + 1 \times (-1)$$

$$= -2 + 10 - 1 = 7$$

   The total is positive, so $H(x) = +1$: the student is predicted to be **placed**. Two stumps out of three said "not placed", but the stump with by far the largest say said "placed", and it wins (Figure 5).

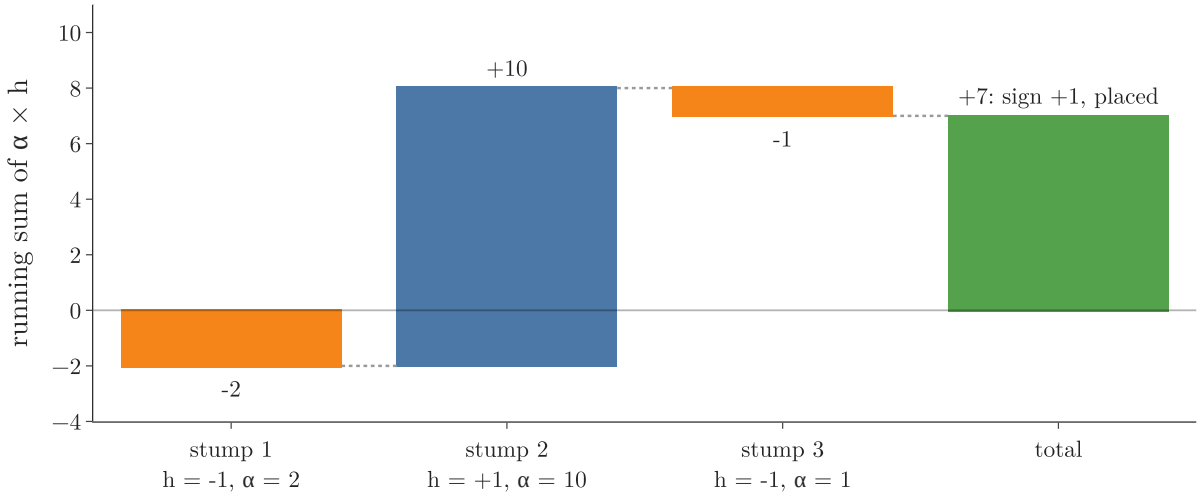


Figure 5: The weighted vote of the example as a running sum. Stump 1 pulls it down by 2, stump 2 lifts it by 10, stump 3 pulls it down by 1; the total, +7, is positive, so the prediction is placed

The pull of a negative vote is why the classes are +1 and -1: a "not placed" vote pulls the total down by its alpha. With 0 and 1, a "not placed" vote would multiply to 0 and could never outweigh a "placed" vote.

6. Why it works: the combined boundary

> **Key point:** Each stump draws one straight cut; their weighted vote draws a staircase boundary that none of them could draw alone.

Each stump on its own is a single straight line, parallel to an axis, and each one gets two students wrong. Lay the three cuts on top of each other (Figure 4, last frame) and they divide the plane into boxes. In each box the weighted vote picks a class.

For our 10 students, the vote of the three stumps is

$$H(x) = \text{sign}\left(0.69\,h_1(x)\right.$$

$$\left.+0.97\,h_2(x) + 1.28\,h_3(x)\right)$$

and it classifies **all 10 correctly** (Figure 6).

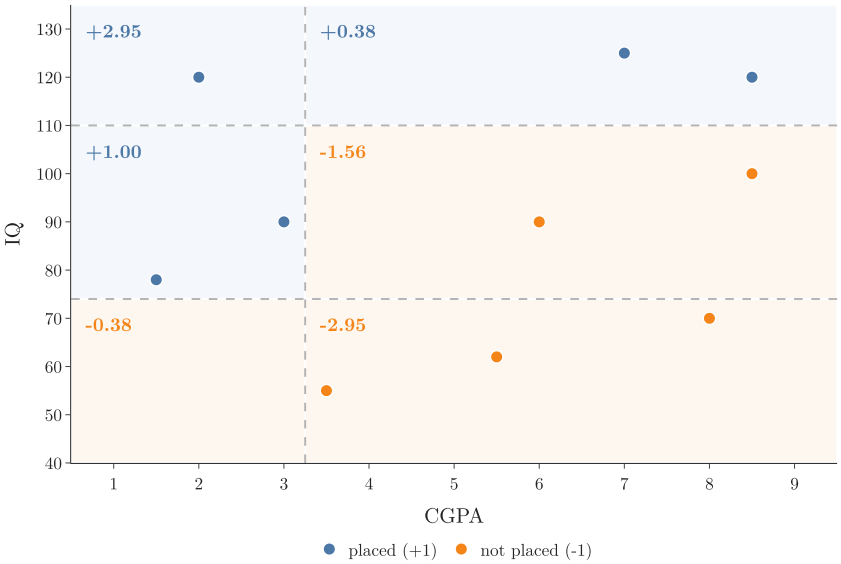


Figure 6: The weighted sum in every box that the three cuts make. Positive boxes (blue) predict placed, negative boxes (orange) not placed. The blue boxes form an upside-down L

The placed region it draws is an upside-down L: low CGPA with an IQ above 74, or any CGPA with an IQ above 110. No single stump can draw that shape, and no single stump got every student right.

With more stages, the boundary can bend in more places and fit more complicated data. From underfitting to overfitting shows that too many stages can also overfit.

7. Summary

Idea	What it means in AdaBoost	Why it matters
Weak learner	each base model is only a little better than guessing	many poor models combine into one strong model
Decision stump	the usual weak learner: a tree with one split	weak on purpose, and easy to read
Labels	+1 and -1	a "not placed" vote pulls the sum down; a 0 would add nothing
Stage-wise additive	stumps are added one at a time; the final model sums them	each new stage can fix the stages before
Mistakes	observations a stump gets wrong become more important for the next stump	the next stump concentrates on them
Alpha	each stump's say in the final vote, larger for fewer mistakes	accurate stumps outvote poor ones
Prediction	$\text{sign}(\sum_t \alpha_t h_t(x))$	the sign turns the weighted sum into a class

- **AdaBoost** (G-167) trains decision stumps one after another; each focuses on the previous stump's mistakes, because those mistakes are made heavier before the next stump is fitted.
- Each stump gets a say, **alpha** (G-192), based on its error: unlike bagging, the votes are not equal, so one stump with a large say can outvote two with small ones.
- The prediction is the sign of the alpha-weighted sum of the stumps' +1/-1 answers.
- Three stumps on 10 students: each makes 2 mistakes, their weighted vote makes none, because their three cuts together draw an upside-down L that no single straight cut can draw. This is how **weak learners** (G-2104) in series become one strong model.

8. Sources

Built from

- CampusX, "How Adaboost Classifier Works? | Geometric Intuition", YouTube, https://www.youtube.com/watch?v=sFKnP0iP0K0
- StatQuest with Josh Starmer, "AdaBoost, Clearly Explained", YouTube, https://www.youtube.com/watch?v=LsK-xG1cLYA

Other references

- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences* 55(1): 119–139. (Conference version: EuroCOLT 1995.)

- Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *CVPR 2001*.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
AdaBoost (Adaptive Boosting) (G-167)	A boosting algorithm that trains simple models (weak learners) one after another on reweighted data, and combines them by a vote in which each model counts by its alpha weight.
Weak learner (G-2104)	A model whose accuracy is only a little better than random guessing.
Strong learner (G-1904)	A model with high accuracy.
Decision stump (G-559)	A decision tree with maximum depth 1: one question, two regions. Too weak alone, it is AdaBoost's usual weak learner.
Stage-wise additive model (G-1867)	A model built as a sum of base models added one at a time, one per stage; boosting builds this kind of model, each new part learning from the mistakes of the ones before.
Alpha (model weight) (G-192)	A base model's say in AdaBoost's final vote; larger when it made fewer mistakes.
Hypothesis function (G-912)	A model written as a function $h(x)$ that maps an input to a prediction.
Sign function (G-1800)	Returns +1 for a positive number and -1 for a negative one; AdaBoost uses it to turn the weighted vote of its stumps into a class.
Boosting (G-318)	Combining many simple models one after another to reduce bias, the error of a model that is too simple.
Random forest (G-1611)	Bagging with decision trees as the base models: many trees, each trained on a random sample of the rows with a random choice of features at every split, vote (classification) or are averaged (regression), which lowers the variance of a single tree.

Term	Meaning
Gradient boosting (G-859)	A way to build a model from many trees added one by one: it starts from a simple guess, and each new tree is trained on the mistakes (pseudo-residuals) of the ensemble so far (a boosting algorithm).
Information gain (G-946)	How much a split reduces the mix of labels: the parent's entropy minus the weighted entropy of its children; the tree splits on the highest.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Upsampling (resampling by weight) (G-2063)	Drawing a new dataset of the same size in which each row is picked with probability equal to its weight, so heavily weighted rows appear more often for the next model.